

## 1. Install required libraries and load the spaCy English model.

```

!pip install spacy pandas matplotlib seaborn emoji
!python -m spacy download en_core_web_sm

```

loading emoji-2.15.0-py3-none-any.whl.metadata (5.7 kB)

requirement already satisfied: spacy-legacy<3.1.0,>=3.0.11 in /usr/local/lib/python3.12/dist-packages  
 requirement already satisfied: spacy-loggers<2.0.0,>=1.0.0 in /usr/local/lib/python3.12/dist-packages  
 requirement already satisfied: murmurhash<1.1.0,>=0.28.0 in /usr/local/lib/python3.12/dist-packages (from spacy-legacy<3.1.0,>=3.0.11)  
 requirement already satisfied: cymem<2.1.0,>=2.0.2 in /usr/local/lib/python3.12/dist-packages (from spacy-legacy<3.1.0,>=3.0.11)  
 requirement already satisfied: preshed<3.1.0,>=3.0.2 in /usr/local/lib/python3.12/dist-packages (from spacy-legacy<3.1.0,>=3.0.11)  
 requirement already satisfied: thinc<8.4.0,>=8.3.4 in /usr/local/lib/python3.12/dist-packages (from spacy-legacy<3.1.0,>=3.0.11)  
 requirement already satisfied: wasabi<1.2.0,>=0.9.1 in /usr/local/lib/python3.12/dist-packages (from spacy-legacy<3.1.0,>=3.0.11)  
 requirement already satisfied: srsly<3.0.0,>=2.4.3 in /usr/local/lib/python3.12/dist-packages (from spacy-legacy<3.1.0,>=3.0.11)  
 requirement already satisfied: catalogue<2.1.0,>=2.0.6 in /usr/local/lib/python3.12/dist-packages (from spacy-legacy<3.1.0,>=3.0.11)  
 requirement already satisfied: weasel<0.5.0,>=0.4.2 in /usr/local/lib/python3.12/dist-packages (from spacy-legacy<3.1.0,>=3.0.11)  
 requirement already satisfied: typer-slim<1.0.0,>=0.3.0 in /usr/local/lib/python3.12/dist-packages (from spacy-legacy<3.1.0,>=3.0.11)  
 requirement already satisfied: tqdm<5.0.0,>=4.38.0 in /usr/local/lib/python3.12/dist-packages (from spacy-legacy<3.1.0,>=3.0.11)  
 requirement already satisfied: numpy>=1.19.0 in /usr/local/lib/python3.12/dist-packages (from spacy-legacy<3.1.0,>=3.0.11)  
 requirement already satisfied: requests<3.0.0,>=2.13.0 in /usr/local/lib/python3.12/dist-packages (from spacy-legacy<3.1.0,>=3.0.11)  
 requirement already satisfied: pydantic!=1.8,!1.8.1,<3.0.0,>=1.7.4 in /usr/local/lib/python3.12/dist-packages (from spacy-legacy<3.1.0,>=3.0.11)  
 requirement already satisfied: jinja2 in /usr/local/lib/python3.12/dist-packages (from spacy-legacy<3.1.0,>=3.0.11) (3.1.6)  
 requirement already satisfied: setuptools in /usr/local/lib/python3.12/dist-packages (from spacy-legacy<3.1.0,>=3.0.11) (75.0.0)  
 requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages (from spacy-legacy<3.1.0,>=3.0.11)  
 requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.12/dist-packages (from spacy-legacy<3.1.0,>=3.0.11)  
 requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from spacy-legacy<3.1.0,>=3.0.11)  
 requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from spacy-legacy<3.1.0,>=3.0.11)  
 requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=3.5.0)  
 requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=3.5.0)  
 requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=3.5.0)  
 requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=3.5.0)  
 requirement already satisfied: pillow>=8 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=3.5.0)  
 requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=3.5.0)  
 requirement already satisfied: annotated-types>=0.6.0 in /usr/local/lib/python3.12/dist-packages (from pydantic!=1.8,!1.8.1,<3.0.0,>=1.7.4)  
 requirement already satisfied: pydantic-core==2.41.4 in /usr/local/lib/python3.12/dist-packages (from pydantic!=1.8,!1.8.1,<3.0.0,>=1.7.4)  
 requirement already satisfied: typing-extensions>=4.14.1 in /usr/local/lib/python3.12/dist-packages (from pydantic!=1.8,!1.8.1,<3.0.0,>=1.7.4)  
 requirement already satisfied: typing-inspection>=0.4.2 in /usr/local/lib/python3.12/dist-packages (from pydantic!=1.8,!1.8.1,<3.0.0,>=1.7.4)  
 requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.8.2)  
 requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests<3.0.0,>=2.13.0)  
 requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests<3.0.0,>=2.13.0)  
 requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests<3.0.0,>=2.13.0)  
 requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from requests<3.0.0,>=2.13.0)  
 requirement already satisfied: blis<1.4.0,>=1.3.0 in /usr/local/lib/python3.12/dist-packages (from thinc<8.4.0,>=8.3.4)  
 requirement already satisfied: confection<1.0.0,>=0.0.1 in /usr/local/lib/python3.12/dist-packages (from thinc<8.4.0,>=8.3.4)  
 requirement already satisfied: click>=8.0.0 in /usr/local/lib/python3.12/dist-packages (from typer-slim<1.0.0,>=0.3.0)  
 requirement already satisfied: cloudpathlib<1.0.0,>=0.7.0 in /usr/local/lib/python3.12/dist-packages (from typer-slim<1.0.0,>=0.3.0)  
 requirement already satisfied: smart-open<8.0.0,>=5.2.1 in /usr/local/lib/python3.12/dist-packages (from typer-slim<1.0.0,>=0.3.0)  
 requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.12/dist-packages (from jinja2 in /usr/local/lib/python3.12/dist-packages)  
 requirement already satisfied: wrapt in /usr/local/lib/python3.12/dist-packages (from smart-open<8.0.0,>=5.2.1)

adding emoji-2.15.0-py3-none-any.whl (608 kB)

608.4/608.4 kB 13.2 MB/s eta 0:00:00

ling collected packages: emoji

Successfully installed emoji-2.15.0

ling en-core-web-sm==3.8.0

loading [https://github.com/explosion/spacy-models/releases/download/en\\_core\\_web\\_sm-3.8.0/en\\_core\\_web\\_sm-3.8.0-py3-none-any.whl](https://github.com/explosion/spacy-models/releases/download/en_core_web_sm-3.8.0/en_core_web_sm-3.8.0-py3-none-any.whl)

12.8/12.8 MB 72.9 MB/s eta 0:00:00

load and installation successful

now load the package via spacy.load('en\_core\_web\_sm')

art to reload dependencies

are in a Jupyter or Colab notebook, you may need to restart Python in

to load all the package's dependencies. You can do this by selecting the

kernel' or 'Restart runtime' option.

```

import spacy
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

```

```
import re
import emoji
from collections import Counter
```

## 2. Load the Twitter US Airline Sentiment dataset.

```
df = pd.read_csv("Tweets.csv")
df.head()
```

|   | tweet_id           | airline_sentiment | airline_sentiment_confidence | negativereason | negativer |
|---|--------------------|-------------------|------------------------------|----------------|-----------|
| 0 | 570306133677760513 | neutral           | 1.0000                       | NaN            |           |
| 1 | 570301130888122368 | positive          | 0.3486                       | NaN            |           |
| 2 | 570301083672813571 | neutral           | 0.6837                       | NaN            |           |
| 3 | 570301031407624196 | negative          | 1.0000                       | Bad Flight     |           |
| 4 | 570300817074462722 | negative          | 1.0000                       | Can't Tell     |           |

Next steps:

[Generate code with df](#)[New interactive sheet](#)

## 3. Select Tweet Text and Sentiment Columns and Remove Missing Values

```
df = df[['text', 'airline_sentiment']]
df.dropna(inplace=True)
df.shape
```

(14640, 2)

## 4. Clean Tweets

```
def clean_tweet(text):
    text = re.sub(r"http\S+|www\S+", "", text) # Remove URLs
    text = re.sub(r"@w+", "", text) # Remove mentions
    text = emoji.replace_emoji(text, replace="") # Remove emojis
    text = re.sub(r"^[^a-zA-Z\s#]", "", text) # Remove special characters
    text = text.lower().strip() # Lowercase
    return text

df['clean_text'] = df['text'].apply(clean_tweet)
df.head()
```

|   | text                                              | airline_sentiment | clean_text                                        |  |
|---|---------------------------------------------------|-------------------|---------------------------------------------------|--|
| 0 | @VirginAmerica What @dhepburn said.               | neutral           | what said                                         |  |
| 1 | @VirginAmerica plus you've added commercials t... | positive          | plus youve added commercials to the experience... |  |
| 2 | @VirginAmerica I didn't today... Must mean I n... | neutral           | i didnt today must mean i need to take another... |  |
| 3 | @VirginAmerica it's really aggressive to blast... | negative          | its really aggressive to blast obnoxious enter... |  |
| 4 | @VirginAmerica and it's a really big bad thing... | negative          | and its a really big bad thing about it           |  |

Next steps: [Generate code with df](#) [New interactive sheet](#)

## 5. Create a Cleaned Tweet Corpus

```
cleaned_corpus = df['clean_text'].tolist()
cleaned_corpus[:5]
```

```
['what said',
 'plus youve added commercials to the experience tacky',
 'i didnt today must mean i need to take another trip',
 'its really aggressive to blast obnoxious entertainment in your guests faces amp they have little recourse',
 'and its a really big bad thing about it']
```

## 6. Initialize the spaCy NLP Pipeline

```
nlp = spacy.load("en_core_web_sm")
```

## 7. Create and Add a Custom spaCy Pipeline Component to Detect Hashtags

```
from spacy.tokens import Doc
from spacy.language import Language

Doc.set_extension("hashtags", default=[])

@Language.component("hashtag_detector")
def hashtag_detector(doc):
    doc._hashtags = [token.text for token in doc if token.text.startswith("#")]
    return doc

nlp.add_pipe("hashtag_detector", last=True)
```

```
hashtag_detector
def hashtag_detector(doc)
<no docstring>
```

## 8. Process the Cleaned Tweets Using Customized spaCy Pipeline

```
docs = list(nlp.pipe(cleaned_corpus))
```

## 9. Extract Lemmas and Part-of-Speech Tags

```
def extract_lemmas_pos(doc):
    return [(token.text, token.lemma_, token.pos_)
            for token in doc
            if not token.is_punct and not token.is_space]

df['tokens'] = [extract_lemmas_pos(doc) for doc in docs]
df.head()
```

|   | text                                              | airline_sentiment | clean_text                                        | tokens                                             |
|---|---------------------------------------------------|-------------------|---------------------------------------------------|----------------------------------------------------|
| 0 | @VirginAmerica What @dhepburn said.               | neutral           | what said                                         | [(what, what, PRON), (said, say, VERB)]            |
| 1 | @VirginAmerica plus you've added commercials t... | positive          | plus youve added commercials to the experience... | [(plus, plus, CCONJ), (you, you, PRON), (ve, v...] |
| 2 | @VirginAmerica I didn't today... Must mean I n... | neutral           | i didnt today must mean i need to take another... | [(i, I, PRON), (did, do, AUX), (nt, not, PART)...  |
| 3 | @VirginAmerica it's really aggressive to blast... | negative          | its really aggressive to blast obnoxious enter... | [(its, its, PRON), (really, really, ADV), (agg...] |
| 4 | @VirginAmerica and it's a                         | ..                | and its a really big bad                          | [(and, and, CCONJ), (a, a, PRON)]                  |

Next steps:

[Generate code with df](#)[New interactive sheet](#)

## 10. Extract Hashtags and Compute Frequencies

```
df['hashtags'] = [doc._.hashtags for doc in docs]

all_hashtags = [tag for tags in df['hashtags'] for tag in tags]
hashtag_freq = Counter(all_hashtags)
hashtag_freq.most_common(10)
```

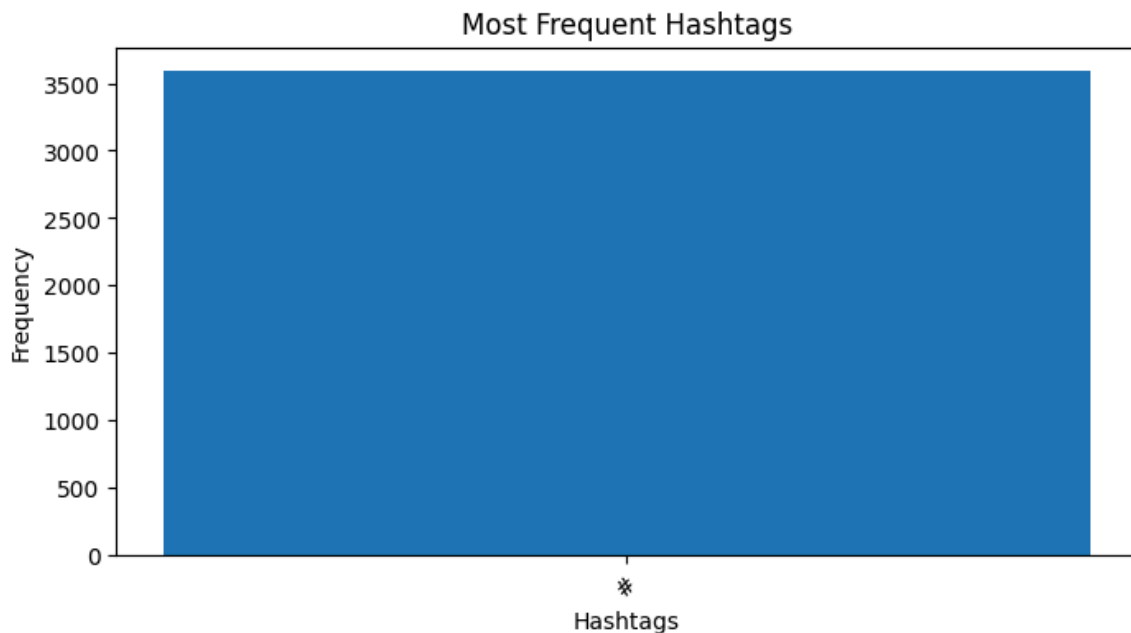
```
[('#', 3590)]
```

## 11. Visualize the Most Frequent Hashtags

```
top_hashtags = hashtag_freq.most_common(10)

if top_hashtags:
    tags, counts = zip(*top_hashtags)

    plt.figure(figsize=(8,4))
    plt.bar(tags, counts)
    plt.title("Most Frequent Hashtags")
    plt.xlabel("Hashtags")
    plt.ylabel("Frequency")
    plt.xticks(rotation=45)
    plt.show()
```



## 12. Filter Negative Tweets and Visualize POS Tag Distribution

```
negative_df = df[df['airline_sentiment'] == 'negative']

pos_tags = []
for tokens in negative_df['tokens']:
    pos_tags.extend([pos for _, _, pos in tokens])

pos_freq = Counter(pos_tags)
pos_df = pd.DataFrame(pos_freq.items(), columns=['POS Tag', 'Count'])
```

```
plt.figure(figsize=(8,4))
sns.barplot(data=pos_df, x='POS Tag', y='Count')
plt.title("POS Tag Distribution in Negative Tweets")
plt.show()
```

